

Laporan Praktikum 6 Kontrol Cerdas

Nama : Levis Mega Nureska

NIM : 224308010

Kelas : TKA – 6A

Akun Github (Tautan) : <https://github.com/levismeganureska>

Student Lab Assistant : Muhammad Fatih Anfa Adhima

1. Judul Percobaan

Judul : Canny Edge Detection & Lane Detection with Instance Segmentation

Tujuan Percobaan

Pada praktikum ini, mahasiswa akan dapat:

- Memahami konsep Canny Edge Detection sebagai metode dasar deteksi tepi.
- Menggunakan Instance Segmentation untuk deteksi jalur rel kereta (Lane Detection).
- Menggunakan dataset Rail Segmentation dari Kaggle untuk eksperimen.
- Menggabungkan metode Canny Edge Detection dengan Instance Segmentation untuk meningkatkan deteksi jalur.

2. Landasan Teori

M Canny Edge Detection, atau deteksi tepi, merupakan langkah penting dalam pengolahan citra yang bertujuan untuk mengidentifikasi batas atau kontur objek dalam gambar. Metode seperti Sobel, Prewitt, dan Roberts telah lama digunakan dalam bidang ini. Namun, Canny Edge Detection dianggap lebih unggul karena mampu mendeteksi tepi dengan akurasi tinggi dan menghasilkan gambar dengan tingkat kebisingan yang lebih rendah (Luh Putu Risma Noviana dkk., 2024). Keunggulan utama Canny Edge Detection dibandingkan metode lainnya adalah kemampuannya dalam **good detection**, yaitu memaksimalkan Signal to Noise Ratio (SNR) sehingga semua tepi dapat terdeteksi dengan baik. Selain itu, metode ini juga memiliki **good location**, yang berarti mampu meminimalkan perbedaan antara lokasi tepi sebenarnya dengan hasil deteksi, sehingga tepi yang terdeteksi lebih akurat (semakin tinggi nilai **Loc**, semakin baik kualitas deteksinya). Keunggulan lainnya adalah **one response to single edge**, yang memastikan bahwa setiap tepi yang terdeteksi adalah tepi yang valid, tanpa adanya duplikasi atau deteksi tepi yang salah (Perangin-angin dkk., 2019). Dalam praktikum ini, Instance Segmentation juga digunakan untuk mendeteksi jalur rel kereta (Lane Detection). Instance Segmentation merupakan proses anotasi atau pemberian label pada suatu gambar, yang biasanya dilakukan menggunakan tools seperti Roboflow (Wulanningrum dkk., 2024). Model pelatihan dalam penelitian ini menggunakan sekitar 2000 gambar rel kereta api dengan berbagai jenis lintasan, termasuk jalur lurus, berbelok, single track, dan double track. Setiap epoch dalam proses pelatihan melibatkan pemasukan dataset ke dalam model, penyesuaian parameter, serta pembaruan bobot agar model dapat mendeteksi dan melakukan segmentasi rel kereta dengan lebih akurat. Proses ini bertujuan untuk mengurangi kesalahan dalam membedakan rel kereta api dengan objek lain berdasarkan anotasi dalam dataset pelatihan. Selain itu, teknik augmentasi data diterapkan untuk meningkatkan kemampuan model dalam mengenali pola yang lebih beragam (Husain dkk., 2024).

Algoritma yang digunakan juga memiliki ketahanan terhadap oklusi kompleks, sehingga sangat berguna dalam adaptasi terhadap lingkungan kendaraan otonom (AV). Algoritma ini telah diintegrasikan dengan berbagai arsitektur jaringan klasifikasi seperti ENet, SegNet, dan ResNet38 yang dirancang khusus untuk identifikasi gambar. Dari hasil penelitian, arsitektur **ResNet38** memiliki akurasi rata-rata tertinggi, tetapi membutuhkan kapasitas memori yang lebih besar dibandingkan metode lainnya (Darwin & Saputro, 2021).

3. Analisis dan Diskusi

Analisis:

1. Seberapa baik deteksi jalur dengan Instance Segmentation dibandingkan Canny?

Deteksi jalur menggunakan **Instance Segmentation** umumnya lebih akurat dibandingkan dengan **Canny Edge Detection** karena mampu mengenali objek secara menyeluruh dan memberikan segmentasi berbasis instance, bukan hanya mendeteksi tepi. Instance Segmentation dapat membedakan jalur dari objek lain dalam gambar, bahkan dalam kondisi pencahayaan yang buruk atau latar belakang yang kompleks. Sebaliknya, Canny Edge Detection hanya mendeteksi perubahan intensitas piksel, sehingga rentan terhadap gangguan seperti bayangan, noise, atau tekstur yang menyerupai tepi jalur..

2. Apakah kombinasi kedua metode dapat meningkatkan akurasi?

Kombinasi **Canny Edge Detection** dan **Instance Segmentation** berpotensi meningkatkan akurasi deteksi jalur. Canny dapat digunakan sebagai langkah awal untuk menyoroti tepi jalur secara kasar, sementara Instance Segmentation dapat mengoreksi hasil deteksi dengan memahami konteks gambar secara lebih mendalam. Integrasi ini dapat mengurangi kesalahan deteksi, terutama dalam kondisi lingkungan yang menantang seperti persimpangan yang rumit atau jalur yang tertutup sebagian..

3. Apa dampak perubahan parameter Canny (thresholds) terhadap hasil deteksi

Perubahan **parameter threshold pada Canny Edge Detection** sangat berpengaruh terhadap hasil deteksi. Jika nilai threshold terlalu rendah, terlalu banyak tepi yang tidak relevan akan terdeteksi, sehingga meningkatkan noise dalam gambar. Sebaliknya, jika threshold terlalu tinggi, beberapa bagian jalur yang lebih samar mungkin tidak terdeteksi, menyebabkan hilangnya informasi penting. Oleh karena itu, pemilihan threshold yang optimal sangat krusial agar Canny Edge Detection dapat memberikan hasil yang lebih akurat, terutama saat dikombinasikan dengan Instance Segmentation.

Diskusi:

Kapan lebih baik menggunakan Canny Edge Detection dibanding Instance Segmentation?

Canny Edge Detection lebih cocok digunakan dalam kondisi di mana perhitungan cepat dan efisiensi komputasi menjadi prioritas, seperti pada perangkat dengan daya pemrosesan terbatas atau sistem real-time yang memerlukan deteksi tepi sederhana tanpa memerlukan segmentasi berbasis objek. Metode ini juga efektif dalam lingkungan dengan kontras tinggi, di mana tepi jalur sangat jelas terlihat, seperti pada citra dengan pencahayaan baik dan latar belakang yang tidak terlalu kompleks. Namun, jika gambar memiliki banyak gangguan atau pencahayaan yang tidak merata, metode ini bisa menghasilkan banyak deteksi yang keliru.

Bagaimana cara meningkatkan deteksi jalur dengan tuning parameter YOLOv8-seg?

Untuk meningkatkan **deteksi jalur menggunakan YOLOv8-seg**, tuning parameter menjadi langkah penting. Beberapa aspek yang dapat disesuaikan termasuk **confidence threshold**, yang menentukan

ambang batas kepercayaan model dalam mendeteksi suatu objek, serta **IoU (Intersection over Union) threshold**, yang mengontrol tingkat tumpang tindih yang diperbolehkan antara prediksi dan ground truth. Selain itu, **penyesuaian arsitektur model** seperti jumlah lapisan konvolusi dan ukuran anchor box dapat membantu model lebih responsif terhadap variasi bentuk dan ukuran jalur rel. Teknik **augmentasi data**, seperti rotasi, pemotongan, dan perubahan pencahayaan, juga dapat meningkatkan kemampuan generalisasi model terhadap berbagai kondisi lingkungan.

Bagaimana metode ini dapat diterapkan dalam sistem navigasi kereta otomatis?

Penerapan metode ini dalam **sistem navigasi kereta otomatis** dapat meningkatkan keamanan dan efisiensi operasional. Instance Segmentation dapat membantu dalam **mendeteksi dan mengenali jalur rel dengan lebih presisi**, memungkinkan sistem untuk memahami struktur lintasan dengan lebih baik, termasuk jalur bercabang, perlintasan sebidang, dan area dengan rintangan. Dengan kombinasi Canny Edge Detection untuk deteksi cepat dan Instance Segmentation untuk pemetaan detail, sistem navigasi dapat mengurangi risiko kesalahan dalam pengambilan keputusan, seperti menghindari tabrakan atau memastikan kereta tetap berada di jalur yang benar. Integrasi metode ini dengan sensor lain seperti LiDAR dan GPS juga dapat meningkatkan akurasi sistem navigasi secara keseluruhan.

4. Assignment

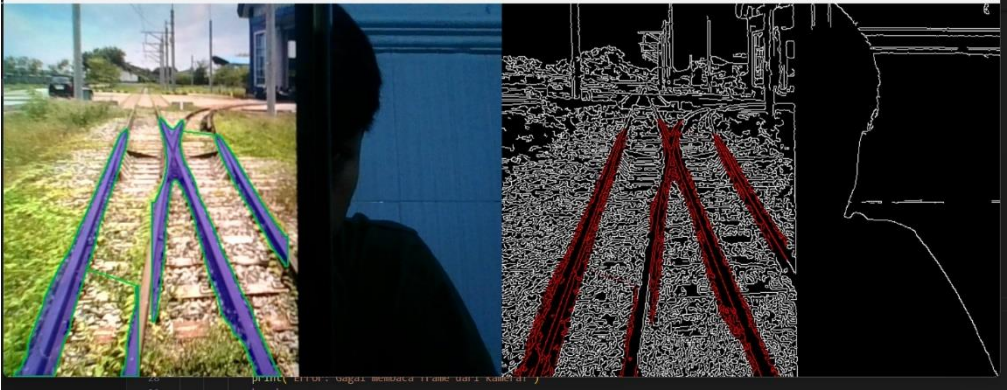
Berdasarkan praktikum keenam yang telah dilakukan dari kode program ini adalah membuat sebuah sistem yang dapat melakukan deteksi tepi menggunakan metode Canny Edge Detection dan mengombinasikannya dengan deteksi objek atau segmentasi menggunakan model YOLO dari Ultralytics untuk menyoroti objek yang terdeteksi dalam sebuah video yang diambil secara real-time dari kamera. Program dimulai dengan memuat model YOLO dari path yang telah ditentukan menggunakan `YOLO(MODEL_PATH)`, memastikan bahwa model tersedia sebelum melanjutkan ke pemrosesan video menggunakan OpenCV. Setelah kamera diakses melalui `cv2.VideoCapture(0)`, setiap frame yang ditangkap dikonversi ke skala abu-abu dengan `cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)` agar dapat dilakukan deteksi tepi menggunakan metode Canny Edge Detection, yang menghasilkan gambar biner dengan tepi putih pada latar belakang hitam melalui `cv2.Canny(gray, low_threshold, high_threshold)`. Secara paralel, frame asli dikonversi ke format RGB dengan `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`, kemudian diproses oleh model YOLO untuk melakukan prediksi objek atau segmentasi, di mana hasil prediksi pertama disimpan dalam `predictions = results[0]`. Jika model menggunakan segmentasi, maka program akan menggambar area objek menggunakan poligon biru transparan dengan `cv2.fillPoly(overlay, [mask], (255, 0, 0))`, sedangkan jika model hanya mendeteksi objek dalam bentuk bounding box, maka objek akan dilingkari dengan kotak hijau menggunakan `cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)`, serta label objek akan ditampilkan jika tersedia menggunakan `cv2.putText()`.

Selanjutnya, masking digunakan untuk membedakan tepi objek yang terdeteksi oleh YOLO dengan tepi yang dideteksi oleh metode Canny, di mana tepi dalam area objek yang dikenali YOLO akan diwarnai merah, sementara tepi lainnya tetap berwarna putih dengan `edges_bgr[np.where((mask_model == 255) & (edges == 255))] = [0, 0, 255]`. Kedua hasil, yaitu frame dengan anotasi YOLO dan hasil deteksi tepi, ditampilkan secara berdampingan menggunakan `np.hstack()` untuk memberikan tampilan visual yang lebih jelas mengenai perbedaan antara tepi umum dan tepi dari objek yang dikenali. Program berjalan dalam loop hingga pengguna menekan tombol 'q', yang akan memicu `cv2.waitKey(1) & 0xFF == ord('q')` untuk keluar, setelah itu kamera akan dilepaskan dengan `cap.release()` dan semua jendela yang terbuka akan ditutup dengan `cv2.destroyAllWindows()` untuk mencegah kebocoran memori. Dengan kombinasi dari metode deteksi tepi tradisional dan teknologi deep learning untuk segmentasi atau deteksi objek, program ini dapat digunakan untuk berbagai aplikasi seperti deteksi rel, pengawasan lalu lintas, dan lainnya.

5. Data dan Output Hasil Pengamatan

Sajikan data dan hasil yang diperoleh selama percobaan. Gunakan tabel untuk menyajikan data jika diperlukan.

No	Variabel	Hasil Pengamatan
1	Canny Edge Detection	
2	Lane Detection dengan Instance Segmentation (YOLOv8-seg)	

3	Menggabungkan Canny Edge Detection dengan Lane Detection	
---	--	--

6. Kesimpulan

1. Program berhasil mengintegrasikan metode **Canny Edge Detection** dengan model **YOLO**, memungkinkan deteksi tepi sekaligus identifikasi dan penyorotan objek dalam video real-time.
2. Model **YOLO** mampu mengenali objek dengan baik, baik dalam bentuk **bounding box** maupun **segmentasi**, yang kemudian ditandai menggunakan warna dan anotasi yang sesuai. Selain itu, teknik **masking** diterapkan untuk membedakan tepi objek yang dikenali YOLO dengan tepi umum hasil Canny Edge Detection, sehingga menghasilkan visualisasi yang lebih jelas.
3. Program dapat berjalan dalam **loop** hingga pengguna menekan tombol keluar, dengan pengelolaan memori yang optimal untuk mencegah kebocoran sumber daya.

7. Saran

Untuk meningkatkan performa dan akurasi, program dapat dioptimalkan dengan **pemanfaatan GPU**, sehingga prediksi YOLO dapat berjalan lebih cepat dan responsif. Selain itu, **preprocessing tambahan** seperti penerapan **Gaussian Blur** sebelum **Canny Edge Detection** dapat membantu mengurangi noise yang tidak diinginkan, sehingga deteksi tepi menjadi lebih akurat.

Agar hasil deteksi lebih mudah dibaca, **tingkat transparansi overlay warna** dapat disesuaikan, sehingga objek yang terdeteksi tetap terlihat jelas tanpa menutupi detail penting dalam gambar. Selain itu, menambahkan fitur **penyimpanan hasil deteksi** dalam bentuk gambar atau video dapat menjadi nilai tambah, memungkinkan analisis lebih lanjut untuk berbagai aplikasi di dunia nyata.

8. Daftar Pustaka

- Darwin, S., & Saputro, N. (2021). SURVEI APLIKASI SEGMENTASI CITRA UNTUK AUTONOMOUS VEHICLE.
- Husain, B. H., Osawa, T., Astiti, S. P. C., Frianto, D., Nandika, M. R., & Agustine, D. (2024). Deteksi Lubang Jalan Secara Otomatis dari Rekaman Drone Menggunakan Model Machine Learning Berbasis YOLOv5 Instance Segmentation di Kota Pekanbaru, Provinsi Riau, Indonesia. *Jurnal Zona*, [https://doi.org/10.52364/zona.v8i2.1268\(2\)](https://doi.org/10.52364/zona.v8i2.1268(2)), 137–146.
- Luh Putu Risma Noviana, I Putu Eka Indrawan, & Gde Iwan Setiawan. (2024). ANALYSIS OF CANNY EDGE DETECTION METHOD FOR FACIAL RECOGNITION IN DIGITAL IMAGE PROCESSING. *Jurnal Manajemen dan Teknologi Informasi*, 15(2), 29–34. <https://doi.org/10.59819/jmti.v15i2.4107>
- Perangin-angin, R., Harianja, E. J. G., & No, J. H. T. (2019). COMPARISON DETECTION EDGE LINES ALGORITMA CANNY DAN SOBEL. 2.
- Wulanningrum, R., Handayani, A. N., & Wibawa, A. P. (2024). Perbandingan Instance Segmentation Image Pada Yolo8. *Jurnal Teknologi Informasi dan Ilmu Komputer*, <https://doi.org/10.25126/jtiik.1148288>

